

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Reminder: Homework Assignment 2

- Due: **11:59 PM** on **Friday, March 13**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
 - Do not upload the data file.
 - Make sure to **rename your notebook file** as instructed.

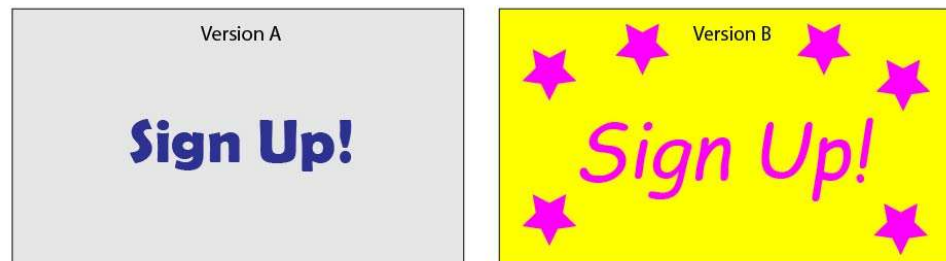
Lecture 14

- Parameter Estimation:
 - Bayesian Parameter Estimation: Updating Beliefs
- Hypothesis Testing

Bayesian Parameter Estimation

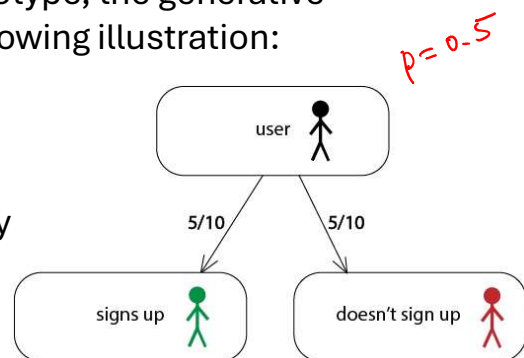
Recall: A/B testing example

- In an A/B test, we are interested in comparing two versions of a webpage.
 - We would like to determine which one performs better at a task X.
- E.g.: Does Version A (new) or Version B (old) perform better at user sign-ups?



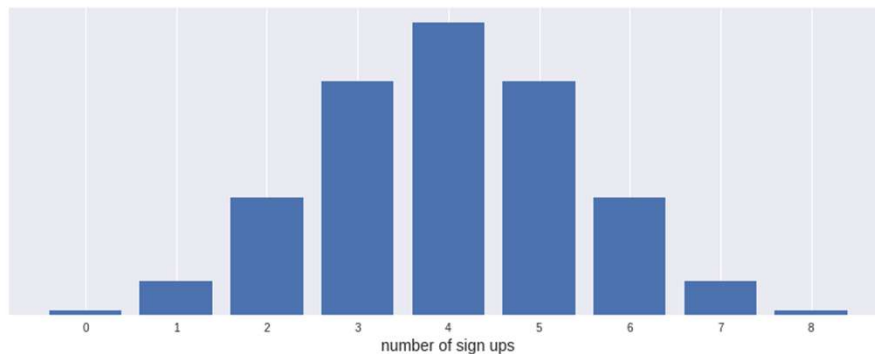
Bayesian parameter estimation

- Consider the generative model for the A/B testing experiment in the previous slide.
- For one person and one webpage prototype, the generative process can be described with the following illustration:
 - Suppose that a user is equally likely to sign up or forgo signing up.
- Say we are selecting ^W8 individuals to try Version A, what does the generative process look like?



The long run distribution

- If one repeats the experiment many times, the graph will represent the binomial distribution.
- The distribution is parameterized by $n = 8$ and $p = 0.5$.



Observations

- We don't know the signup rates p_A and p_B for Version A and B a priori.
- Clearly, we can use the maximum likelihood (ML) to estimate the parameters.
 - However, the sample is small, and random events can greatly skew estimation.
- Confidence interval tells us where the interval may lie with 95% confidence.
 - However, what we are often interested in is the probability of individual values.
- There is no way to encode our belief about the parameter.
 - Say we know a parameter should be ~ 0.9 because another research showed that.
 - How can we incorporate that in estimating the parameter?

```
# Suppose outcomes are
```

```
outcomes = [True] * 5 + [False] * 3 ← 5 sign-ups / 8  
outcomes                                     = 0.625
```

```
[True, True, True, True, True, False, False, False]
```

```
bootstrap = np.random.choice(outcomes, size=8, replace=True)  
bootstrap_proportion = sum(bootstrap)/len(bootstrap)  
bootstrap_proportion
```

```
0.75
```

```
bootstrap_proportions = []  
for _ in range(10_000):  
    bootstrap = np.random.choice(outcomes, size=8, replace=True)  
    bootstrap_proportion = sum(bootstrap)/len(bootstrap)  
    bootstrap_proportions.append(bootstrap_proportion)  
np.percentile(bootstrap_proportions, (2.5, 97.5)) ← 95% CI  
array([0.25, 0.875])
```

```
bootstrap = np.random.choice([True, False], size=8, replace=True)  
bootstrap_proportion = sum(bootstrap)/len(bootstrap)  
print(bootstrap)  
bootstrap_proportion
```

```
[False True False False False True True True]
```

```
0.5
```

```
bootstrap_proportions = []  
for _ in range(10000):  
    bootstrap = np.random.choice([True, False], size=8, replace=True)  
    bootstrap_proportion = sum(bootstrap)/len(bootstrap)  
    bootstrap_proportions.append(bootstrap_proportion)  
np.percentile(bootstrap_proportions, (2.5, 97.5))
```

```
array([0.125, 0.875])
```

Running the simulation backwards

- Rather than generate random outcomes of the experiment, we record random parameters that could have yielded the observed results.
- Which parameters could have generated 5 signups in 8 trials?
 - Recall that p is continuous probability between 0 and 1.

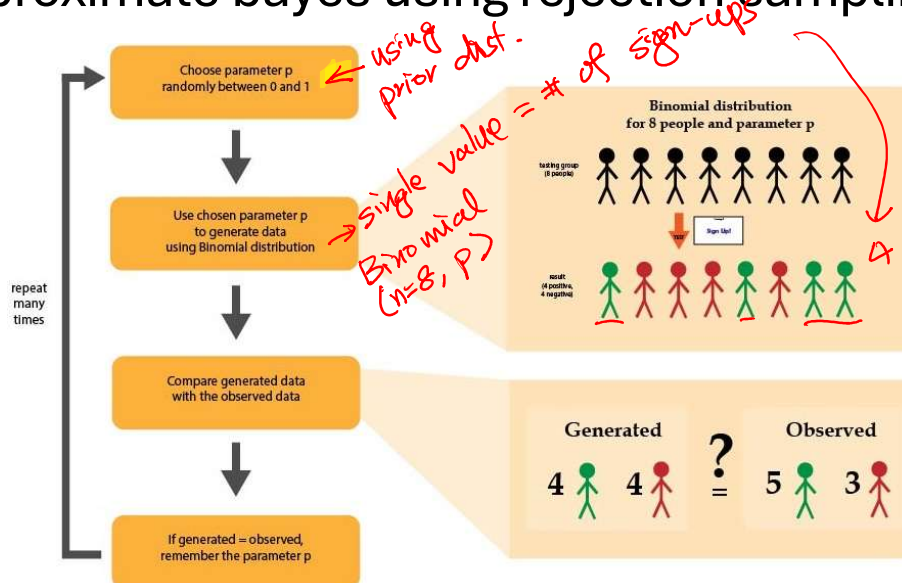
Bayesian inference using rejection sampling

- For simplicity, we will only consider Version A.
- How can we determine the most likely value of p_A parameter for Version A?
- For that we will use a simple algorithm for Bayesian inference called **rejection sampling**.
 - We simulate parameters and only retain those that agree with our data.

Rejection sampling

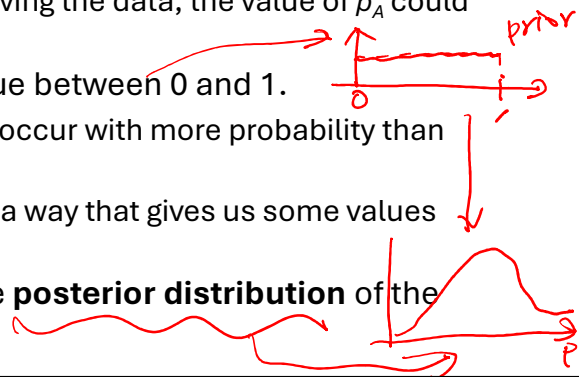
1. Sample a parameter value from the set of possible values of p_A .
 - This is called a *prior* and represents our prior belief on p_A .
 - In our example, we believe that p_A is any possible value in $[0, 1]$ continuum.
2. Generate an outcome, given that parameter value (in our example, run the test that is described by Binomial distribution with parameter p_A and $n = 8$).
3. • If the generated outcome agrees with the observed data, record the value of the parameter p_A as a possible value that can generate the observed results.
 - If the generated outcome is not the observed data, ignore (or reject) the value of the parameter p_A .
4. Repeat the procedure many times.

Approximate bayes using rejection sampling



Exploring the distribution of the parameter p

- What does the distribution of p_A look like?
 - This distribution represents the values that most likely generate the observed data.
 - I.e., what we believe, based on observing the data, the value of p_A could have been.
- Our prior belief was that p_A is any value between 0 and 1.
 - Results show that some values of p_A occur with more probability than other values.
 - We could have encoded our belief in a way that gives us some values more often than others.
- The resulting distribution is called the **posterior distribution** of the parameter p_A .



```
# Version A
n_A = 8          # number of people Version A was tested on
obs_A = 5        # observed number of signups for Version A
p_A_recorded = []

num_of_recorded_vals = 100_000

for _ in range(num_of_recorded_vals):
    # Sample a parameter value p_A
    p_A = np.random.uniform(0, 1)

    # For the p_A, generate an outcome from the Binomial distribution
    v_A = np.random.binomial(n_A, p_A)

    # Compare generated outcomes to the observed data
    if v_A == obs_A:
        p_A_recorded.append(p_A)

len(p_A_recorded)
```

11076

Handwritten notes:

- data* (next to `obs_A = 5`)
- prior* (with an arrow pointing to `np.random.uniform(0, 1)`)
- 8* (under `n_A`)
- # of accepted p_A values* (with an arrow pointing to the output `11076`)


```
plt.style.use(["seaborn-v0_8-darkgrid"])
```

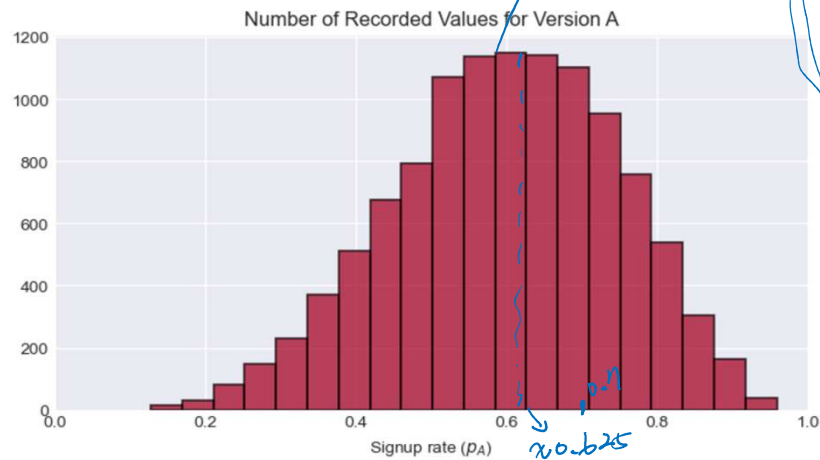
```
fig = plt.figure(figsize=(8, 4))
```

```
plt.hist(p_A_recorded, bins=20, alpha=0.75, color="#A60628", edgecolor='black', linewidth=1.2)
```

```
plt.xlim(0, 1)
```

```
plt.xlabel("Signup rate ( $p_A$ )")
```

```
plt.title("Number of Recorded Values for Version A")
```



Analyzing the simulation results

- Histograms shows the probability of p_A to be highest around 0.625.
- This agrees with the likelihood.
- This shows that the value could also be, with non-negligible probabilities, in other values.
- In this case, this does not add much to the likelihood.
- Imagine, however, that we have another dataset of signups for version A.
 - E.g., out of 9 users tested, 6 signed up for version A.

$$\frac{6}{9} \approx 0.666\ldots$$

$$\approx 0.667$$

Updating our beliefs

- We previously estimated p_A and obtained a posterior distribution for p_A .
- Instead of sample p_A uniformly from $[0, 1]$, we can now sample it from this posterior distribution.
 - In the next round of inference, the posterior from the previous step becomes the new prior.
- Practically, we can sample from the histogram that represents the posterior distribution.

```
np.random.choice(p_A_recorded)
```

```
0.687093743654701
```

new_data

```
# Version A
n_A = 9          # number of people Version A was tested on
obs_A = 6        # observed number of signups for Version A
p_A_recorded_experiment_2 = []

num_of_recorded_vals = 100_000

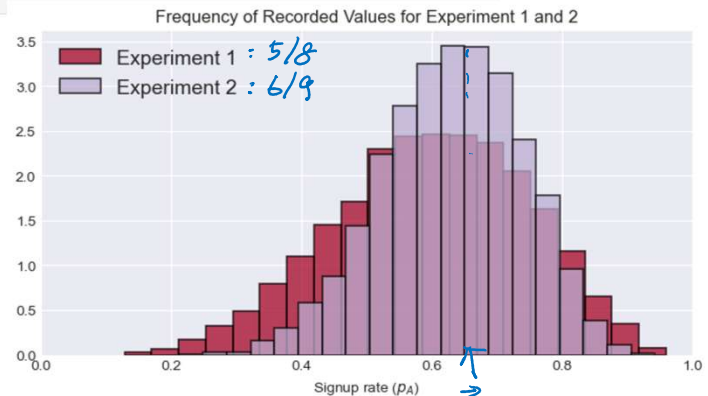
for _ in range(num_of_recorded_vals):
    # Sample a parameter value  $p_A$  from the posterior
    p_A = np.random.choice(p_A_recorded)

    # For the  $p_A$ , generate an outcome from the Binomial distribution
    v_A = np.random.binomial(n_A, p_A)

    # Compare generated outcomes to the observed data
    if v_A == obs_A:
        p_A_recorded_experiment_2.append(p_A)
```

```
plt.figure(figsize=(8, 4))

plt.hist(p_A_recorded, bins=20, density=True, alpha=0.75,
        label="Experiment 1", color="#A60628", edgecolor='black', linewidth=1.2,)
plt.hist(p_A_recorded_experiment_2, bins=20, density=True, alpha=0.75,
        label="Experiment 2", color="#bcadd1", edgecolor='black', linewidth=1.2)
plt.xlim(0, 1)
plt.xlabel("Signup rate ($p_A$)")
plt.title("Frequency of Recorded Values for Experiment 1 and 2")
plt.legend(loc='upper left', fontsize=14)
```



```
# Version A
n_A = 8          # number of people Version A was tested on
obs_A = 6        # observed number of signups for Version A
p_A_recorded_experiment_3 = []

num_of_recorded_vals = 100_000

for _ in range(num_of_recorded_vals):
    # Sample a parameter value p_A from the posterior
    p_A = np.random.choice(p_A_recorded_experiment_2)

    # For the p_A, generate an outcome from the Binomial distribution
    v_A = np.random.binomial(n_A, p_A)

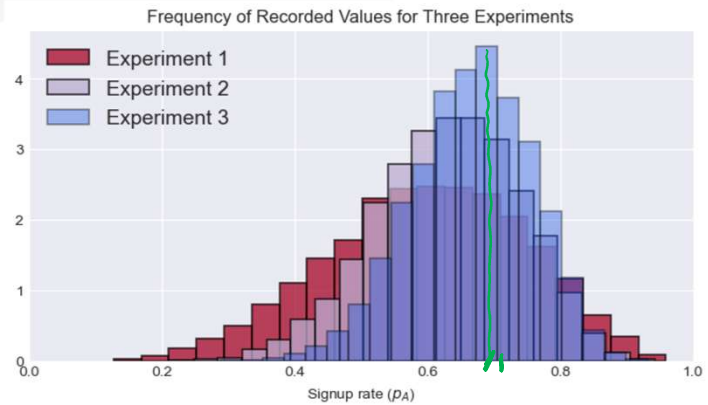
    # Compare generated outcomes to the observed data
    if v_A == obs_A:
        p_A_recorded_experiment_3.append(p_A)
```

new data {

$6/8 = 0.75$

```
plt.figure(figsize=(8, 4))
```

```
plt.hist(p_A_recorded, bins=20, density=True, alpha=0.75,
        label="Experiment 1", color="#a60628", edgecolor='black', linewidth=1.2)
plt.hist(p_A_recorded_experiment_2, bins=20, density=True, alpha=0.75,
        label="Experiment 2", color="#bcadd1", edgecolor='black', linewidth=1.2)
plt.hist(p_A_recorded_experiment_3, bins=20, density=True, alpha=0.4,
        label="Experiment 3", color="#1f5ae6", edgecolor='black', linewidth=1.2)
plt.xlim(0, 1)
plt.xlabel("Signup rate ($p_A$)")
plt.title("Frequency of Recorded Values for Three Experiments")
plt.legend(loc='upper left', fontsize=14);
```



```
from scipy.stats import gaussian_kde
```

```
kde1 = gaussian_kde(p_A_recorded, bw_method=0.8)
kde2 = gaussian_kde(p_A_recorded_experiment_2, bw_method=0.8)
kde3 = gaussian_kde(p_A_recorded_experiment_3, bw_method=0.8)
```

```
x = np.linspace(0, 1, 1000)
```

```
# Plot KDE for each experiment
```

```
fig = plt.figure(figsize=(10, 6))
```

```
plt.plot(x, kde1(x), label="Experiment 1", color="#a60628", linewidth=2)
plt.plot(x, kde2(x), label="Experiment 2", color="#bcadd1", linewidth=2)
plt.plot(x, kde3(x), label="Experiment 3", color="#1f5ae6", linewidth=2, alpha=0.75)
```

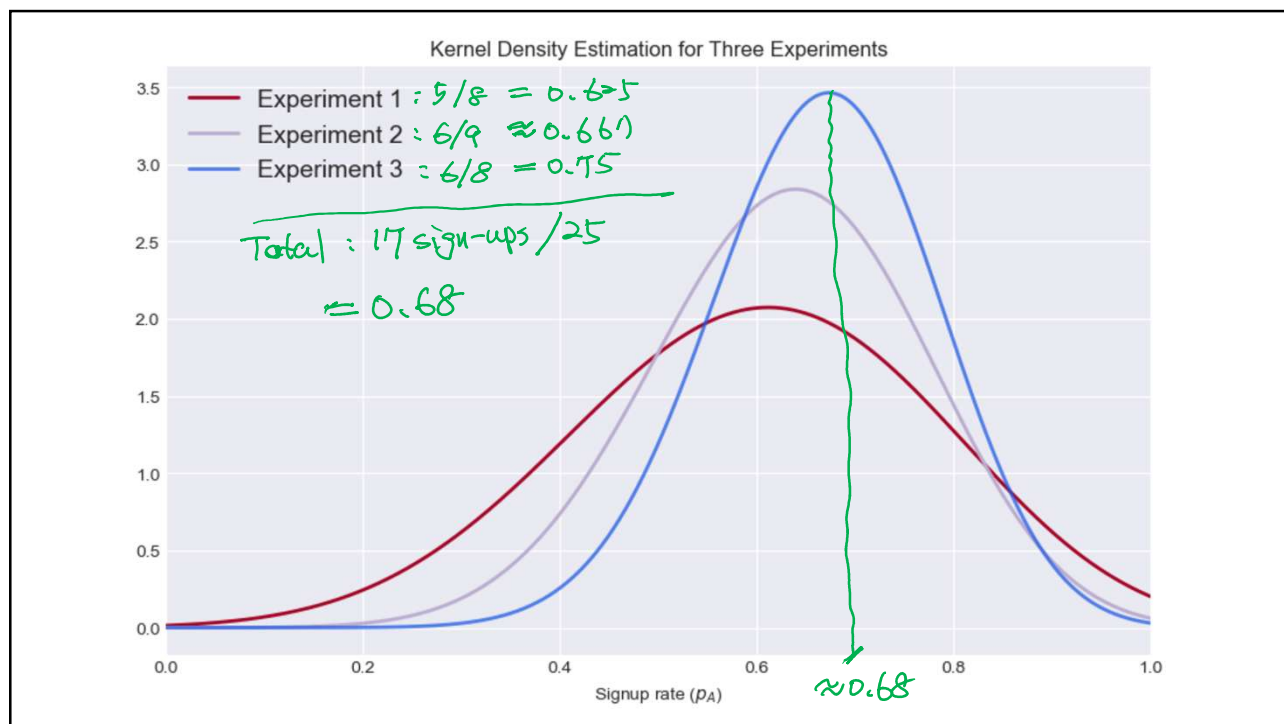
```
plt.xlim(0, 1)
```

```
plt.xlabel("Signup rate ($p_A$)")
```

```
plt.title("Kernel Density Estimation for Three Experiments")
```

```
plt.legend(loc='upper left', fontsize=14)
```

```
plt.style.use(["seaborn-v0_8-darkgrid"])
```



Bayesian inference for parameter estimation

- Bayesian inference uses probability theory to estimate model parameters by treating them as random variables and representing uncertainty about their values.
 - Encodes prior beliefs:** Prior knowledge is expressed with a prior distribution.
 - Updates with data:** Observed data combine with the prior to produce a posterior distribution.
 - Accumulates evidence:** The posterior can become the new prior as more data are observed.

$$p(\text{param} | \text{data}) = \frac{p(\text{data} | \text{param}) p(\text{param})}{p(\text{data})}$$

likelihood prior
 posterior

Hypothesis Testing

Hypothesis testing

- Definitions:
 - A **statistic** is a numerical value or measure calculated directly from a **sample data**.
 - Summarizes or describes characteristic of the sample.
 - Often used to estimate or infer information about a population.
 - A **parameter** is a numerical value that describes some aspect of the **entire population**.
 - A fixed value (often unknown) that would be obtained if data from the complete population were available.

Hypothesis testing (cont'd.)

- Hypothesis is an assumption or a statement about a population parameter (e.g., the population mean or proportion).
- The goal of a hypothesis test is to evaluate evidence from sample data to decide between two competing hypotheses.
- The difference between parametric and non-parametric methods:
 - Parametric methods make assumptions about the parameters and the underlying distributions.
 - Non-parametric methods do not make any assumptions about underlying parameters and their distributions.

Hypothesis testing questions

In hypothesis testing, we are typically interested in:

1. The magnitude of the difference between groups or hypotheses
 - This is referred to as the *effect size*.
 - Example: Economists have shown that universal basic income (UBI) can increase happiness.
 - The results may be statistically significant, but how large is the effect?
 - If participants are only 1% happier, is it worth spending ~\$3 trillion (the expected annual cost) on UBI?

Hypothesis testing questions (cont'd.)

2. The probability associated with the observed difference

- How variable is the measured or inferred difference?
- How likely is it that the observed difference is due to chance alone?
- This probability is quantified as the **p-value**.

Data in hypothesis testing

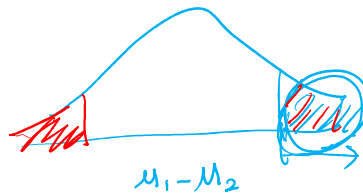
- Can involve data from either one or two samples. For instance:
 - Comparing a single sample to a population with a known parameter.
 - E.g., you use a new math curriculum on a sample of randomly selected students and compare the results to the national average.
 - Does your curriculum improve math exam scores (compared to the national average)?
 - Comparing two samples to determine whether there is a difference between them.
 - E.g., you teach your math curriculum to a group of students in the morning and another group of students in the afternoon.
 - Is there a difference between learning math in the morning or the afternoon?

Terminology

- We are typically testing two hypotheses.
 - The first hypothesis assumes that there is no change in our system.
 - Any changes that we observe are merely due to natural variance in the population.
 - This is referred to as the **null hypothesis (H_0)**.
 - The second hypothesis assumes the differences we are observing are not likely to arise due to chance alone.
 - By "not likely," we mean something that would probabilistically be rare in occurrence.
 - This is referred to as the **alternative hypothesis (H_a)**.

Terminology (cont'd.)

- The alternative hypothesis can be:



1. Single-tailed

- Our new curriculum improves the average score compared to the national average: $\mu_{\text{new_curriculum}} > \mu_{\text{national_average}}$
- A new medication decreases cholesterol levels:

$$\mu_{\text{cholesterol_treatment}} < \mu_{\text{cholesterol_control}}$$

2. Two-tailed

- The values must be different from a given value.
- Example: Rate of prescription of opioids in Hawaii is different from the national average. This implied:

$$\mu_{\text{opioids_hawaii}} < \mu_{\text{opioids_nationally}} \text{ or } \mu_{\text{opioids_hawaii}} > \mu_{\text{opioids_nationally}}$$

About the hypothesis tests

- Hypotheses (H_0 and H_a) are mutually exclusive.
 - For example, there is a difference or there is no difference.
 - But not: the difference is greater than 0 or greater than 10.
- A null hypothesis can never be proven to be true.
 - Failing to find evidence for the alternative hypothesis only makes the null plausible.
 - We say that we fail to reject the null hypothesis or that the study did not find sufficient evidence to reject the null hypothesis.
- We will say that the results are statistically significant when they are unlikely to be due to chance alone.

Example: Data from a theoretical distribution

- Suppose the data come from two normal distributions:
 - Means: $\mu_1 = 232$ and $\mu_2 = 219$
 - Standard deviations: $\sigma = 12$ (same for both distributions)
- For example, this could represent the average ad time in minutes viewed daily on YouTube versus Vimeo.
 - We have 90 days worth of viewing data.

```

np.random.seed(42)

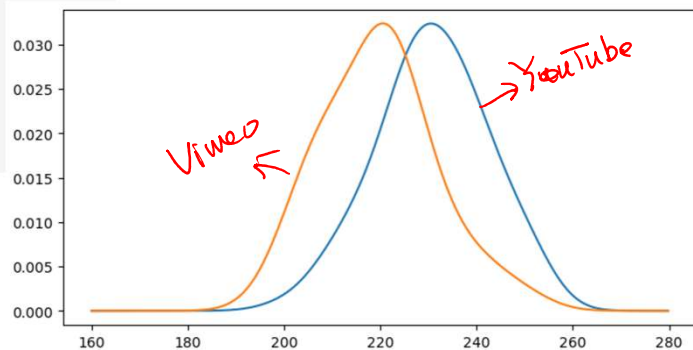
youtube = np.random.normal(232, 12, 90)
vimeo = np.random.normal(219, 12, 90)

kde_1 = sp.stats.gaussian_kde(youtube, bw_method=0.5)
kde_2 = sp.stats.gaussian_kde(vimeo, bw_method=0.5)

# Estimate kernel density for the support (x-axis)
x_values = np.arange(160, 280, 0.25)
x_densities_1 = kde_1.evaluate(x_values)
x_densities_2 = kde_2.evaluate(x_values)

plt.figure(figsize=(8, 4))
plt.plot(x_values, x_densities_1)
plt.plot(x_values, x_densities_2)
plt.show()

```



Assessing the variability of our statistic: Different samples

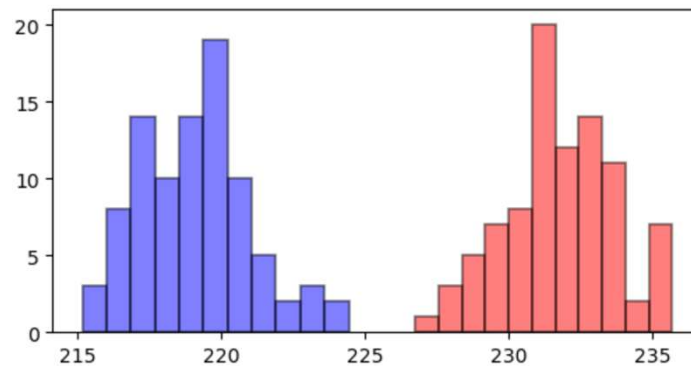
- The difference between the means is a useful statistic to assess the difference between both samples.
 - We call a statistic computed on the observed data the **observed statistic**.
- Given that the data consists of random samples, collecting new data will undoubtedly lead to a change in observed statistic.
- We need a distribution of the difference between the two samples to compare our statistic.
 - This is the distribution of the test statistic under the null hypothesis.

```
np.mean(youtube) - np.mean(vimeo)
```

```
11.159994107248792
```

Assessing the variability of our statistic: Different samples (cont'd.)

- We will start with an example where the samples are clearly (visually) different.
 - To do so, we generate a dataset with $\sigma = 2$.



Question

- What is the mean of the distribution of the difference between the means?

```
youtube = np.random.normal(232, 2, 90)
vimeo = np.random.normal(219, 2, 90)
means_difference.append(np.mean(youtube) - np.mean(vimeo))
```

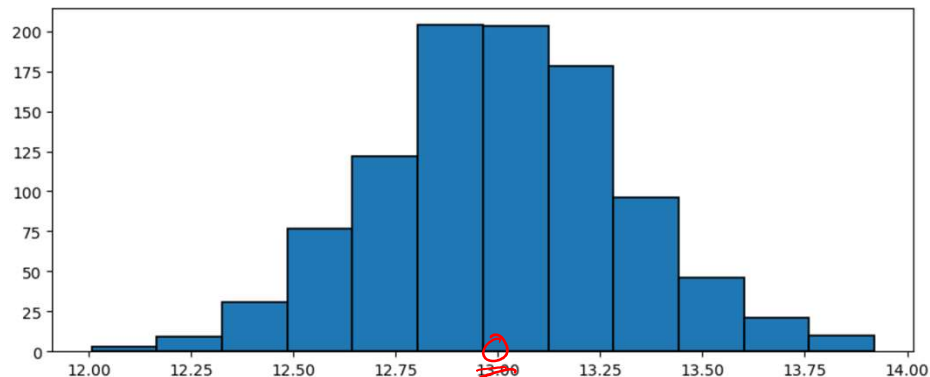
```

means_difference = []

for _ in range(1000):
    youtube = np.random.normal(232, 2, 90)
    vimeo = np.random.normal(219, 2, 90)
    means_difference.append(np.mean(youtube) - np.mean(vimeo))

plt.figure(figsize=(8, 4))
plt.hist(means_difference, edgecolor='k', linewidth=1.2, bins=12)
plt.show()

```



What if samples are similar?

- What does the difference between two samples of the same distribution look like on average?

```

youtube_1 = np.random.normal(232, 2, 90)
youtube_2 = np.random.normal(232, 2, 90)
means_difference.append(np.mean(youtube_1) - np.mean(youtube_2))

```

```
means_difference = []

for _ in range(1000):
    youtube_1 = np.random.normal(232, 2, 90)
    youtube_2 = np.random.normal(232, 2, 90)
    means_difference.append(np.mean(youtube_1) - np.mean(youtube_2))

plt.figure(figsize=(8, 4))
plt.hist(means_difference, edgecolor='k', linewidth=1.2, bins=12)
plt.show()
```

